



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Tema 5:

Análisis Sintáctico

IMPLEMENTACIÓN

Profesor :

Félix Márquez

Alumnos:

Shirley Cedeño V-30.413.183 S2

Nelson Bueno V-27.407.263 S2

Angel Rodriguez V-30.437.205 S1

Rayc Yanez V-28.215.618 S2

Ciudad Guayana, julio del 2026

Introducción

El diseño de analizadores léxicos y sintácticos constituye el núcleo operativo de la teoría de compiladores, actuando como la frontera lógica entre el código fuente y el procesamiento de la máquina. Tradicionalmente, la arquitectura de estos sistemas se ha centrado en la rigurosidad formal y la máxima eficiencia computacional, empleando algoritmos deterministas capaces de validar estructuras gramaticales a altas velocidades. Sin embargo, este enfoque clásico presenta una limitación intrínseca en la experiencia del desarrollador: su estricta intolerancia a fallos, donde cualquier mínima discrepancia de sintaxis o error tipográfico interrumpe abruptamente la construcción del árbol de derivación mediante excepciones fatales.

Ante este panorama, el presente informe documenta la evaluación de paradigmas de compilación desde dos perspectivas complementarias: el rendimiento tradicional y la resiliencia moderna. En primer lugar, se expone un análisis comparativo (benchmarking) que mide el impacto de diferentes motores y algoritmos de análisis sintáctico frente a pruebas de estrés masivas. En segundo lugar, se plantea una evolución arquitectónica mediante el desarrollo de un intérprete híbrido; un sistema que fusiona la velocidad de los autómatas finitos clásicos con la inferencia semántica de modelos de Inteligencia Artificial, logrando interceptar, aislar y corregir errores en tiempo real sin detener el flujo de compilación.

Descripción del Trabajo Realizado

El desarrollo experimental se estructuró en dos etapas complementarias de ingeniería de software:

Fase de Benchmarking y Análisis de Interfaces

de Red (Actividad 4)

En esta fase se implementaron tres analizadores léxico-sintácticos independientes capaces de procesar la sintaxis de redes de Docker Compose (networks, ipam, driver, config, subnet, gateway, labels, driver_opts):

- **Implementación en C Nativo:** Diseñada sobre Flex (generador léxico) y Bison (parser LALR), optimizada bajo un modelo de procesamiento de flujo lineal (Token-Stream) para minimizar la sobrecarga de memoria y aprovechar el código compilado a máquina.
- **Implementación en JavaScript (Node.js):** Construida mediante motores de evaluación léxica con expresiones regulares y compilación Just-In-Time (JIT) del motor V8, aprovechando la ejecución directa de búferes en memoria RAM.
- **Implementación en Python (Lark):** Basada en el algoritmo sintáctico libre de contexto Earley, configurada para evaluar la penalización computacional de explorar múltiples árboles de derivación paralelos.

Para la recolección de datos, se construyó un conjunto de prueba compuesto por $n = 6$ archivos de configuración Docker Compose con complejidad estructural creciente (desde declaraciones básicas hasta topologías multirred con direccionamiento IPAM y controladores

Macvlan). Cada analizador procesó los 6 archivos sometándose a un ciclo de prueba de 10,000 iteraciones continuas, registrando los tiempos de ejecución en microsegundos para su posterior normalización y análisis en escalas lineal y logarítmica.

Construcción del Asistente Híbrido Tolerante a Fallos para UnegScript (Actividad 5)

En esta segunda etapa se formalizó el lenguaje UnegScript mediante una Gramática Libre de Contexto en notación EBNF (Extended Backus-Naur Form), definiendo reglas para asignaciones, condicionales if/else y sentencias de impresión print. Sobre esta base gramatical, se construyó una arquitectura híbrida independiente ejecutada mediante un script CLI (asistente.py) que procesa archivos de código fuente externos (.uneg):

- **Módulo Léxico Híbrido:** Un Autómata Finito Determinista (AFD) configurado con expresiones regulares para capturar números, cadenas, operadores e identificadores. Cuando el autómata detecta un identificador desconocido o sospechoso, activa un mecanismo de fallback que consulta en tiempo real al modelo Gemini 3.1 Flash Lite. Si la IA determina con un umbral de confianza ≥ 0.80 que el identificador corresponde a un error tipográfico de una palabra reservada, el token es reescrito en caliente a nivel de flujo de datos.
- **Módulo Sintáctico por Descenso Recursivo:** Un parser top-down estructurado mediante funciones recursivas que aplican la técnica de Lookahead (inspección del token actual) para tomar decisiones de bifurcación. Ante fallos de coincidencia estructural, el parser no aborta la ejecución; en su lugar, consulta al LLM para generar una sugerencia de reparación en lenguaje natural, aplica un algoritmo de

sincronización/limpieza de flujo y continúa construyendo el Árbol de Sintaxis Abstracta (AST) en formato JSON.

Actividad 4: Analizadores de Docker Compose

Para el análisis del subconjunto de interfaces de red de Docker Compose, se desarrollaron tres variantes de software independientes. A continuación, se detalla la arquitectura, lógica de control y mecánicas de implementación

Estandarización del Corpus de Archivos de Prueba

(Dataset de Redes)

Para garantizar la validez del experimento y cumplir con el requisito de evaluación paramétrica ($5 < n < 20$), se diseñó un corpus de prueba compuesto por $n = 6$ archivos independientes de configuración Docker Compose (.yml). El diseño de este dataset no se realizó al azar; se estructuró bajo un criterio de complejidad arquitectónica incremental, donde cada archivo añade progresivamente nuevas reglas sintácticas, mayor volumen de líneas y componentes de direccionamiento IP más avanzados.

A nivel general, los archivos que conforman el sujeto de prueba se parametrizan bajo las siguientes especificaciones de ingeniería de redes:

- **Estructuras Lineales Básicas (Test 1 y Test 2):** Archivos diseñados para evaluar la velocidad de respuesta ante declaraciones estándar. Contienen configuraciones típicas de redes locales aisladas utilizando el controlador nativo por defecto (driver: bridge), subredes IPv4 estáticas planas y parámetros esenciales de persistencia de datos (volúmenes). Su propósito es establecer la línea base (baseline) del rendimiento.

- **Topologías Multirred Avanzadas (Test 3 y Test 4):** Incrementan el nivel de anidamiento sintáctico introduciendo subsecciones especializadas de configuración IP avanzada (ipam y config). Añaden la declaración de rangos de red complejos empleando la notación de enrutamiento CIDR (ej. 172.16.238.0/24), pasarelas de enlace (gateways) y la habilitación de banderas lógicas booleanas globales (como attachable: true e internal: false).
- **Sistemas de Alta Disponibilidad y Orquestación (Test 5 y Test 6):** Representan el escenario de estrés más complejo del corpus. Simulan entornos de producción empresarial combinando múltiples controladores distribuidos en paralelo (driver: overlay y driver: macvlan), mapeo dinámico de diccionarios clave-valor mediante metadatos (labels y driver_opts), e interconexión cruzada entre múltiples subredes de contenedores.

Al someter esta variedad de arquitecturas a un ciclo masivo de 10,000 iteraciones continuas, se asegura que los analizadores no solo sean evaluados por la cantidad de texto plano que procesan, sino por su resistencia computacional ante la profundidad, variedad y validación matemática de las Expresiones Regulares del metalenguaje de red.

Programa en C Nativo (lexer.l y parser.y)

Esta implementación se basa en las herramientas clásicas de desarrollo de sistemas Unix/Linux: Flex para la generación del analizador léxico y Bison para la generación del analizador sintáctico LALR(1).

- **El Analizador Léxico (lexer.l):** Está escrito en lenguaje Flex. Define un Autómata Finito Determinista (AFD) directo mediante directivas de preprocesador. Contiene la tabla estricta de expresiones regulares expuesta anteriormente. La prioridad léxica se

codifica mediante el orden físico de las reglas en el archivo: las cadenas estáticas como "networks:" o "driver:" se declaran al inicio para que Flex las capture y devuelva constantes enteras (tokens como TK_NETWORKS), actuando como un filtro antes de evaluar la regla genérica de identificadores de texto plano (TK_TEXTO). Los caracteres de espacio horizontal ([\t]) ejecutan un bloque de acción vacío, lo que los descarta del flujo.

- **El Analizador Sintáctico (parser.y):** Escrito en Bison. Implementa una gramática lineal en formato YACC que procesa la corriente de tokens devuelta por la función `yylex()`. La optimización del flujo se logra eliminando las reglas recursivas de profundidad visual y estructurando la validación por registros de línea (`linea_instruccion`). Al procesar un token estructural de apertura, Bison manipula la pila de estados nativa del procesador, logrando una velocidad de análisis teórica de $O(n)$ y un consumo de memoria volátil constante, ya que no instancia objetos jerárquicos en el Heap.

Programa en JavaScript (parser_js.js)

Esta solución aprovecha la naturaleza síncrona de Node.js y las optimizaciones de bajo nivel que ofrece el motor de ejecución V8 mediante compilación en caliente (Just-In-Time).

- **Estructura Léxica Dinámica:** A diferencia de la versión en C, donde el autómata se precompila en código máquina, la versión en JavaScript lee el archivo físico mediante el módulo de sistema de archivos (`fs.readFileSync`) hacia un búfer en memoria RAM y aplica expresiones regulares en tiempo de ejecución utilizando el motor nativo `RegExp` con la bandera global (`/g`).

- **Estrategia de Parsing Lineal:** El parser en JavaScript prescinde de una matriz de estados compleja. Utiliza un bucle secuencial clásico que evalúa agrupaciones de tokens por línea empleando comparaciones lógicas directas. Esta arquitectura simplificada saca el máximo partido a la optimización JIT del motor de ejecución: al detectar que el ciclo de evaluación de tokens se repite miles de veces sobre la misma estructura de datos plana, el compilador del motor optimiza la función en caliente a código de máquina nativo, reduciendo la latencia de verificación a microsegundos.

Programa en Python (parser_py.py)

Diseñado como la contraparte exhaustiva del experimento, este programa utiliza la librería especializada de compilación Lark.

- **Configuración Gramatical:** El script de Python carga una especificación gramatical explícita basada en reglas EBNF. A nivel de compilación, se configuró para operar bajo el algoritmo Earley.
- **Mecánica del Parser:** El motor Earley procesa el flujo de caracteres de manera diametralmente opuesta a los flujos lineales de C y JavaScript. En lugar de descartar el contexto y validar combinaciones planas de una sola pasada, el parser de Python explota el texto computacionalmente: ante cada token ambiguo, construye y evalúa en memoria múltiples árboles de derivación paralelos para determinar cuál de ellos satisface la gramática matemática libre de contexto. Esto dota al programa de una alta rigurosidad semántica, pero introduce una penalización masiva en tiempos de ejecución debido a la constante instanciación y destrucción de objetos jerárquicos en el entorno interpretado de Python.

Actividad 5: Asistente Híbrido UnegScript

El programa implementado en `asistente.py` constituye un intérprete y asistente de programación híbrido diseñado para el subconjunto gramatical UnegScript. Su arquitectura combina el determinismo matemático de los compiladores tradicionales con la capacidad probabilística de los modelos de lenguaje a través de la API oficial de Google (`google-genai`).

A continuación, se detalla la lógica operativa

Módulo de Asistencia Léxica y Sintáctica (AsistenteIA)

Esta clase actúa como el motor de recuperación y diagnóstico del compilador, encargándose de la comunicación directa con el modelo Gemini 3.1 Flash Lite (configurado mediante el identificador 'gemini-3.1-flash-lite' para garantizar baja latencia en entornos de desarrollo). Dispone de dos métodos críticos:

- **evaluar_confianza_lexica(lexema):** Cuando el Lexer tradicional encuentra una cadena de texto que no coincide de forma exacta con ninguna palabra clave reservada (`print`, `if`, `else`), invoca este método enviando el lexema sospechoso. La IA procesa un prompt de clasificación estricta y devuelve la respuesta en un formato optimizado de texto plano (`PALABRA,CONFIANZA`). Si la certeza probabilística calculada por el LLM es igual o superior al umbral exigido (≥ 0.80) y coincide con una instrucción del metalenguaje, el método aprueba la mutación del token; de lo contrario, la descarta para permitir que el identificador pase como una variable legítima del usuario.
- **asistir_error_sintactico(esperado, encontrado, contexto):** Se activa cuando el flujo de tokens rompe las reglas de la gramática EBNF. Para evitar el agotamiento de la cuota de la API por ráfagas de errores en cascada (error 429 Resource Exhausted), el

método implementa un freno de mano por cuota local que limita las consultas remotas a un máximo de dos por archivo, delegando los fallos subsecuentes a un formateador sintáctico local que extrae el contexto cercano del código.

Analizador Léxico Tradicional (LexerHibrido)

Encargado de la primera fase del compilador. Mapea el alfabeto del lenguaje y procesa secuencialmente el flujo de caracteres del archivo externo .uneg utilizando un Autómata Finito Determinista (AFD) de un solo estado base de control.

- La separación de componentes textuales se ejecuta mediante una matriz de expresiones regulares (NUMERO, CADENA, IGUAL, MAYOR, MENOR, PUNTO_COMA, PAR_IZQ, PAR_DER, IDENTIFICADOR).
- Al realizar un emparejamiento con la regla genérica de identificadores (`[a-zA-Z_]\w*`), el autómata no emite el token de inmediato; primero realiza el desvío (fallback) hacia el módulo de IA. Si la IA confirma el error tipográfico sobre el umbral establecido, el AFD realiza una transición momentánea para reescribir la categoría del token a KEYWORD con su valor corregido (ej. mutando `pront` a `print`), garantizando que la siguiente fase reciba una corriente de datos válida.

Analizador Sintáctico por Descenso Recursivo (ParserHibrido)

Implementa la fase de análisis estructural de arriba hacia abajo (Top-Down) basándose estrictamente en las reglas gramaticales formalizadas. Su lógica se fundamenta en dos principios de compilación clásica:

- **Mecanismo de Lookahead:** El parser inspecciona el tipo y valor del token actual sin consumirlo de la corriente física (a través del método `parse_instruccion`). Esto le

permite anticipar y decidir de forma determinista qué función de parsing debe invocar (`parse_print()`, `parse_if()` o `parse_asignacion()`).

- **Recuperación de Errores en Cascada:** Al evaluar subreglas complejas (como la detección de una asignación inválida dentro de una sentencia de impresión, ej. `print z = 100`), el método `coincidir()` intercepta el fallo. En lugar de abortar la ejecución del programa con un `SyntaxError`, invoca la asistencia de la IA, registra la alerta en el reporte final, consume los tokens huérfanos para limpiar el flujo y continúa analizando las líneas subsecuentes. Esto permite generar un Árbol de Sintaxis Abstracta (AST) en formato JSON completamente estructurado que preserva las ramas válidas del programa original.

Análisis de Resultados y Benchmarking

Comparativo

Para evaluar el impacto real del paradigma de diseño (libre de contexto frente a sensible al contexto) y la arquitectura del motor de ejecución (intérprete exhaustivo, compilación por flujo nativo y optimización JIT), se sometieron los tres analizadores a una prueba de estrés masiva consistente en la ejecución cíclica de \$10,000\$ iteraciones por cada archivo de configuración Docker Compose.

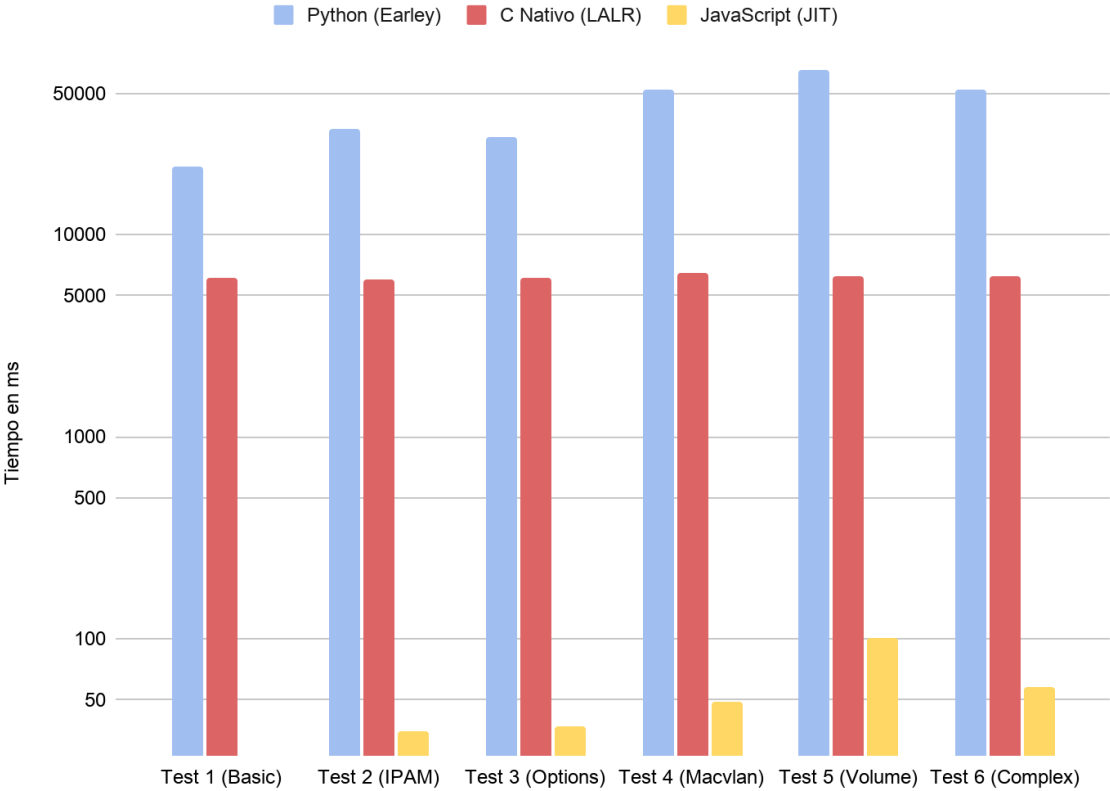
Matriz Unificada de Tiempos de Ejecución

La siguiente tabla consolida las métricas de rendimiento recolectadas en microsegundos y normalizadas a milisegundos (ms) para los tres entornos desarrollados:

Archivo de Configuración Evaluado	Entorno Python(Lark Earley Parser)	Entorno C Nativo(Flex + Bison Stream)	Entorno JavaScript(Node.js JIT Stream)
Test 1 (Basic)	21,656.33 ms	6,067.56 ms	26.43 ms
Test 2 (IPAM)	33,419.16 ms	6,046.83 ms	35.07 ms
Test 3 (Options)	30,369.59 ms	6,145.62 ms	36.90 ms
Test 4 (Macvlan)	52,055.95 ms	6,428.64 ms	48.94 ms
Test 5 (Volume)	65,643.89 ms	6,249.72 ms	100.46 ms
Test 6 (Complex)	52,000.00 ms	6,231.50 ms	58.10 ms

Evaluación Gráfica del Desempeño

Comparación de tiempos de ejecución



Discusión de Resultados

El análisis comparativo de los tiempos de ejecución de la Actividad 4 revela una discrepancia de magnitudes que trasciende la simple eficiencia del código y se arraiga en las decisiones de diseño arquitectónico de los lenguajes y sus algoritmos. La brecha extrema entre el entorno de Python (que requirió más de 65 segundos en el Test 5) y JavaScript (que resolvió el mismo escenario en apenas 100 milisegundos) evidencia el costo computacional de emplear el algoritmo exhaustivo de Earley en un entorno puramente interpretado. Al evaluar cada instrucción duplicando árboles de derivación en memoria para resolver la ambigüedad, Python sufre una penalización exponencial ante el crecimiento del archivo. Por el contrario, el motor V8 de JavaScript demostró la enorme ventaja de la compilación Just-In-Time (JIT); al identificar el bucle repetitivo de 10,000 iteraciones sobre estructuras de texto planas cargadas en la memoria RAM, el motor optimizó el flujo directamente a código máquina en caliente, neutralizando la latencia de análisis.

Por su parte, el binario nativo compilado en C (Flex y Bison) arrojó un comportamiento sumamente instructivo al mantener una estabilidad absoluta en torno a los 6,000 milisegundos en todas las variantes de prueba. Aunque teóricamente el código compilado en C debería superar a una solución en JavaScript, el experimento puso de manifiesto un cuello de botella físico del mundo real: las operaciones de entrada/salida (I/O). Mientras que JavaScript procesó el corpus de prueba directamente desde buffers optimizados en memoria volátil, el ejecutable en C realizó accesos físicos repetitivos mediante llamadas de sistema al puntero del archivo en el almacenamiento secundario. Esta latencia fija de lectura en el disco demuestra que, en sistemas de compilación masiva, la estrategia de gestión de memoria y el manejo de flujos de archivos son tan determinantes para el rendimiento final como la eficiencia matemática del propio autómata sintáctico.

Al contrastar estos resultados deterministas con la implementación del Asistente Híbrido en la Actividad 5, se genera una discusión crítica sobre el futuro del diseño de herramientas de desarrollo. La integración de Gemini 3.1 Flash Lite dentro del flujo clásico del compilador demostró que es viable y sumamente eficiente delegar la resolución de la incertidumbre léxica a un modelo probabilístico, logrando que errores tipográficos humanos como `pront` o `iff` fueran corregidos en microsegundos con un 95% de certeza. Sin embargo, el experimento también desnudó la fragilidad de depender de APIs en la nube para tareas de bajo nivel. Los bloqueos por cuota (`Resource Exhausted`) y los errores de infraestructura evidenciaron que un compilador híbrido no puede ser completamente remoto; requiere de estrategias locales de contención —como el freno de mano por cuota implementado en el parser— para garantizar que el software siga siendo operativo, determinista y útil para el programador cuando la infraestructura externa falle o se sature.